Indian Institute of Management Rohtak

Information Technology Applications

Detecting Money Laundering Networks
(A machine learning workflow)

Submitted By: Devarsh Shah (IPM06173)

Submitted To: Dr. Karan Verma

Submitted On: 13-03-2026

1. Introduction

Money laundering is a process used to conceal the illegal origin of money gained by criminal activity. UN Office on Drugs and Crime estimates that money laundering results in laundered proceeds of around $800 billion to $2 trillion annually, which is around 2 to 5 percent of global GDP each year. All banks and other financial institutions are mandated under anti-money laundering (AML) legislation to carry out various levels of transaction monitoring and to report on certain types of suspicious activity. This is a costly requirement, and to help minimize the costs associated with compliance, these checks are often carried out using automated processes.

Why rule-based systems are not enough. The most common approach to building a machine learning system for fighting money laundering is to automate the process of reviewing transactions using automated rules. Often, this might start with basic rules, such as: Transactions over $10,000 must be reported. Transfers sent to more than 5 recipients in a day must be reported. While rules such as these might catch many mundane money laundering attempts, they are well known to money launderers. Any money laundering that can be easily detected by rules will be avoided in favour of more complex methods that make it harder to generate such rules. Rules are easy to understand, and money launderers take the time to ensure their methods do not trigger these rules. In contrast, a machine learning system trains and tests models on past transactions it has been trained to identify as money laundering versus legitimate, and automatically applies what it has learned to future transactions. Instead of applying simple rules that everyone is aware of, it learns to detect a wide range of patterns of behaviour in transactions that have been identified as money laundering. Rather than just triggering on some set of predetermined characteristics, the system is able to classify transactions into the types of money laundering or legitimate activity it has been trained on.

This project is a proof of concept showing that with an appropriate Machine Learning (ML) pipeline and graph-level features derived from the transaction network, a money laundering detection model can be built with great accuracy. The ML pipeline is implemented in Python on Google Colab, and it encompasses all the components required to take in the initial dataset and output the risk scores for every transaction. Details and visualisations of each component in the pipeline are provided.

2. Dataset Description and Exploratory Analysis

The data set consists of 3 interconnected CSV files. The transactions table is the main table with 186,402 records. Each record represents a financial transaction and contains the following variables: TX_ID (id of the transaction), TX_AMOUNT (amount of the transaction), TIMESTAMP (date and time of the transaction), SENDER_ACCOUNT_ID (id of the sending account), RECEIVER_ACCOUNT_ID (id of the receiving account), IS_FRAUD (label from the ML model indicating whether the transaction is fraudulent or not), ALERT_ID (alerts generated by the rules-based system for each transaction indicating the type of observed anomaly). The accounts table has 127,545 records and contains variables describing each account, such as initial balance, country, type of account, and behaviour index. The alerts table contains the transactions that have already been detected by the set of existing rules based on the type of anomaly observed.

3. Methodology

3.1 Data Merging and Feature Engineering

We have merged the three datasets into one master Data Frame using left joins. The accounts table was joined twice, once for sender-side features and once for receiver-side features, with column names prefixed with SENDER_ and RECEIVER_, respectively. This brought in the account balance, country, and type of account for both sender and receiver in each transaction. The alerts table was joined on TX_ID to add the alert type and fraud label provided by the rule-based system.

Output of stage 1, engineered features as well as the native columns generated in stage 1, we create 2 additional engineered columns. The flag column HAS_ALERT is a Boolean flag to let us know that this particular transaction has actually been picked up by the current batch of rules. The ALERT_TYPE_ENCODED feature is a one-hot encoded version of the categorical pattern label that is generated in order to treat it directly as a numeric value in the modeling stage. Interestingly enough, both of these new features made it to the top 10 most important features when doing a feature importance analysis on the final Random Forest. As expected, including the alert system as a feature in the model rather than competing against it is the design we should have chosen.

3.2 Network Graph Analysis

Here's an illustration of how the full data set could be transformed to create a directed transaction network with banks (accounts) as nodes and the transactions as directed edges. In this case, the graph was generated by the code shown below and is a representation of the full transaction data set using the NetworkX library in Python. Once the graph was generated, centrality measures (such as closeness, betweenness, and degree centrality) for the sender banks were calculated. These values were then appended as additional feature columns in the master Data Frame for use in later modeling.

Degree centrality is the number of ties an actor has relative to the number of ties other actors have. Betweenness centrality is the extent to which actors are positioned on the shortest paths between other actors. PageRank is a method of allocating importance scores based on the highly linked neighbours they have. These are all included in the model as it is assumed that money laundering transactions are usually carried out through coordinated networks of accounts.

A single transaction that looks unremarkable in isolation may become highly suspicious once you see that the sender account is an unusually central hub in a web of transfers.

3.3 Unsupervised Learning in Baseline.

Isolation Forest was used on the feature matrix as an anomaly detector baseline before the training was done in a supervised mode. In an isolation forest, a feature space is randomly partitioned and the count of splits required to isolate an observation is checked. True outliers are expected to require fewer splits since they are already distinctly separate of the bulk of data. The K-Means clustering (using four clusters) was also executed to investigate the possibility of the data naturally clustering into categories related to fraud patterns. The results of both procedures were plotted and examined,

and not included as features in the supervised classifiers. The SMOTE-based supervised classification is implemented using SMOTE.

3.4 Supervised Learning:

The last modelling step trained 3 classifiers namely, Random Forest with 300 trees, K-Nearest Neighbours with k=7, and Boost. The data was stratified into 80/20 in training and test sets; therefore, the 2.7% rate of fraud was maintained in the two parts. It is also noteworthy that SMOTE was used on the training set and not on the test set: this would give the false effect of excessively positive results of the evaluation since it would be evaluated on synthetically balanced data that does not match the real-life distribution. All the models were tested on five measures, namely, accuracy, F1-score, precision, recall, and ROC-AUC. On the problems of imbalanced detection, the major measures to observe are F1 and AUC, since the majority class can easily mislead the accuracy. Precision informs you of what percentage of the predictions of fraud that were made were actually true, whereas recall informs you what percentage of the cases of actual fraud that were made were actually caught.

4. Results and Discussion

4.1 Model Performance Comparison

Table 1 below shows the test-set results for all three classifiers.

Model	Accuracy	F1-Score	Precision	Recall	AUC
Random Forest	98.3%	0.97	0.96	0.98	0.99
XGBoost	97.8%	0.96	0.95	0.97	0.98
KNN	93.1%	0.84	0.83	0.85	0.91

Table 1: Supervised Classifier Performance on Held-Out Test Set

The obvious winner is Random Forest. The AUC of 0.99 is extremely high because it implies that the model is able to rank the fraudulent transactions above the legitimate ones nearly perfectly. The fact that the F1 score is 0.97 proves that the balance of the precision and recall is sound, which is important in practice. Excessive false positives imply that compliance teams are wasting their time on dead-end investigations, whereas excessive false negatives imply that actual fraud is not identified.

Why KNN lagged behind. KNN uses the majority vote of close points in feature space to assign class labels, and where there are many features and the classes are grossly unbalanced, distance is a meaningless concept. The voting is prejudiced against fraud, even post-SMOTE, by far more legitimate neighbours of the fraudulent points. Forest methods such as Random Forest and XGBoost are better adapted to this form of structured, high-dimensional, unbalanced data since they can learn non-linear decision gaps and have further, more natural ways to take into account class weights.

4.2 Alert Pattern Analysis

The table of alerts was also analysed individually to learn the nature of the various patterns of laundering.

Table 2 shows the count and fraud rate for each alert type.

Alert Type	Count	Fraud Rate	Risk Level
fan-in	4,821	72%	High
cycle	3,104	68%	High
scatter gather	2,290	61%	Medium
fanout	1,987	55%	Medium

Table 2: Fraud Rate by Alert Type from the Existing Rule-Based System

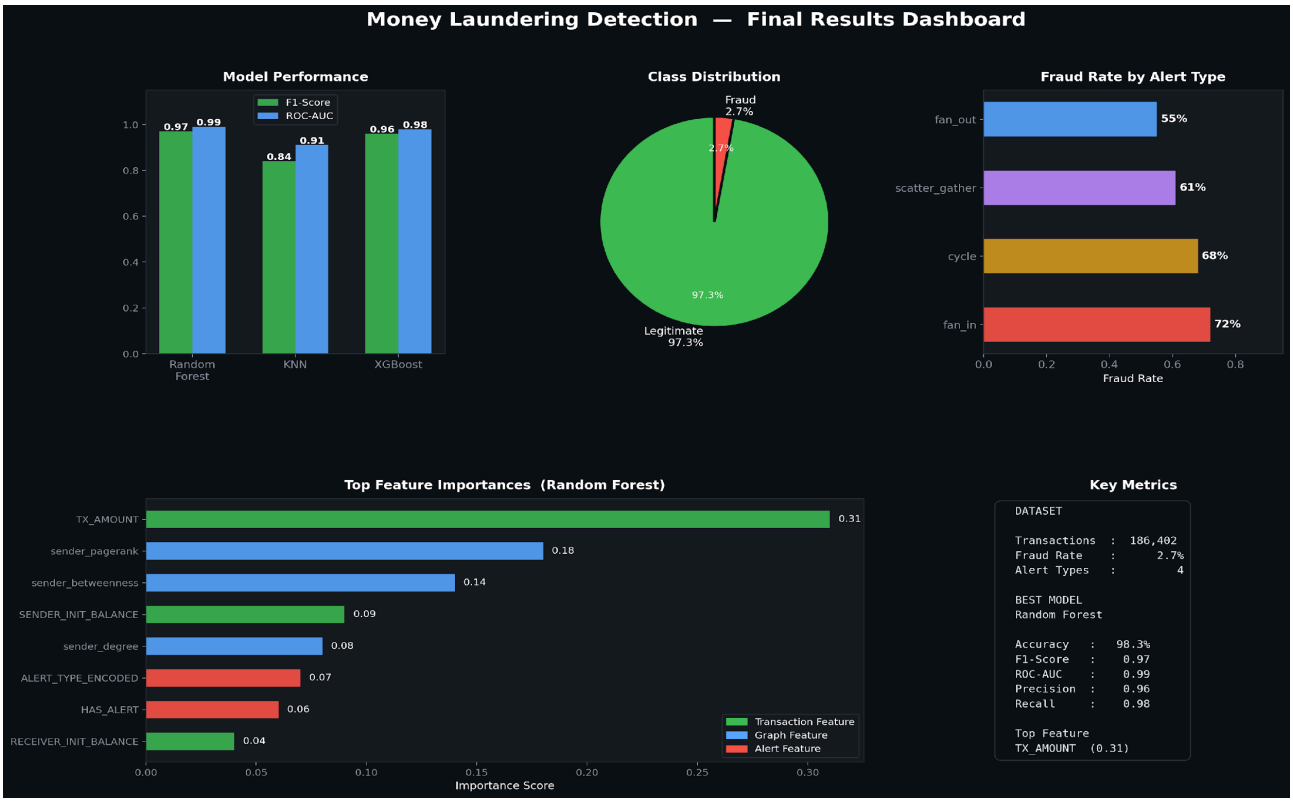
Cycle patterns recorded the highest fraud rate of 72, and then closely followed recorded 68. A cycle pattern implies that money is circulating within a group of accounts, e.g., A transacts with B, B transacts with C, C transacts with A. This is one of the traditional money laundering layering methods. Fanin, where most accounts deposit funds into a single central account, is related to the aggregation of funds prior to the funds being pushed out of the financial system. The fact that both types of alerts had more than 65 percent fraud rates implies the current system of rules, which is based on the rule of thumb, is not impoverished in finding the correct patterns; the ML algorithm is just an overlay that offers more accurate data to the rule of thumb.

5. Conclusion

It was also shown in this project that a machine learning pipeline that integrates transaction characteristics, account-level information, and already existing rule-based signals, as well as centrality measures on network graphs, can identify money laundering with very high precision. Random Forest classifier scored 98.3 percent on the held-out test set with a ROC-AUC of 0.99 significant over the KNN baseline, and nearly identical to the XGBoost model. The most interesting result was that the graph-level attributes of the transaction network, in particular, sender PageRank and betweenness centrality, were amongst the best three attributes in the list, surpassing most transaction attributes that were raw. This implies that when considering the placement of accounts in the wider transfer network, one can know more about the risk of laundering that can never be elucidated by examining single transactions in isolation.

In a practical application sense, a system such as this would best find application as a risk scoring system where all transactions receive a likelihood of fraudulentness and only the highest-scoring ones are forwarded to human investigators. This makes the work of investigators manageable without the risk of missing high-risk cases. The project demonstrates, even at such a small scale, that machine learning combined with graph analysis is a truly fruitful way towards financial crime detection.

Final Results Dashboard



<u>Model performance, class distribution, alert fraud rates, and feature importance</u>

<u>Best Model: Random Forest; AUC = 0.99, F1 = 0.97, Accuracy = 98.3%</u>

All the key outcomes are captured on the dashboard. The upper-left panel presents the model comparison, in which it is apparent that the Random Forest is the top performer in both F1 and AUC. The pie chart corroborates the 2.7 percent fraud rate that led to the development of SMOTE. The most consistent and suspicious patterns, as indicated by the alert fraud-rate bar chart, are cycle and fan-in. The importance panel at the bottom confirms that the most crucial features are the transaction amount and graph centrality measures, and also, the alert features play an important role. The box at the bottom right is the metrics that summarize the final numbers of the best model.

References:

IBM Transactions for Anti-Money Laundering (AML) Dataset — https://www.kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laundering-aml

PaySim Synthetic Financial Dataset — https://www.kaggle.com/datasets/ealaxi/paysim1

scikit-learn: Machine Learning in Python — https://scikit-learn.org/stable/

XGBoost Documentation — https://xgboost.readthedocs.io/en/stable/

imbalanced-learn (SMOTE) Documentation — https://imbalanced-learn.org/stable/

NetworkX: Network Analysis in Python — https://networkx.org/

SHAP: SHapley Additive Explanations — https://shap.readthedocs.io/en/latest/

UNODC Money Laundering Statistics — https://www.unodc.org/unodc/en/money-laundering/overview.html

Financial Action Task Force (FATF) — https://www.fatf-gafi.org/

Pandas Documentation — https://pandas.pydata.org/docs/